

GitHub MCP Server Installation Guide for OpenCode

Complete Setup, Configuration, Authentication, and Usage

Purpose: This guide walks you through installing and configuring the official GitHub MCP server for OpenCode, enabling AI-powered GitHub operations directly from your OpenCode sessions.

Table of Contents

1. [Overview](#)
 2. [What You'll Get](#)
 3. [Prerequisites](#)
 4. [Installation Methods](#)
 5. [Authentication Setup](#)
 6. [OpenCode Configuration](#)
 7. [Verification & Testing](#)
 8. [Available Tools](#)
 9. [Real-World Usage Examples](#)
 10. [Troubleshooting](#)
 11. [Best Practices](#)
 12. [Advanced Configuration](#)
 13. [Multiple GitHub Accounts](#)
 14. [Security Considerations](#)
 15. [FAQ](#)
-

Overview

What is the GitHub MCP Server?

The **GitHub MCP (Model Context Protocol) Server** is an official integration that connects OpenCode to GitHub's REST and GraphQL APIs, allowing you to:

- Create, read, update issues and pull requests
- Search code across repositories
- Manage branches, commits, and releases
- Review and comment on pull requests
- Create and manage repositories
- Work with GitHub Actions and workflows
- And much more!

Why Use It?

WITHOUT GitHub MCP:

You: "Create a PR for this feature"

OpenCode: "I can't access GitHub. Please do it manually."

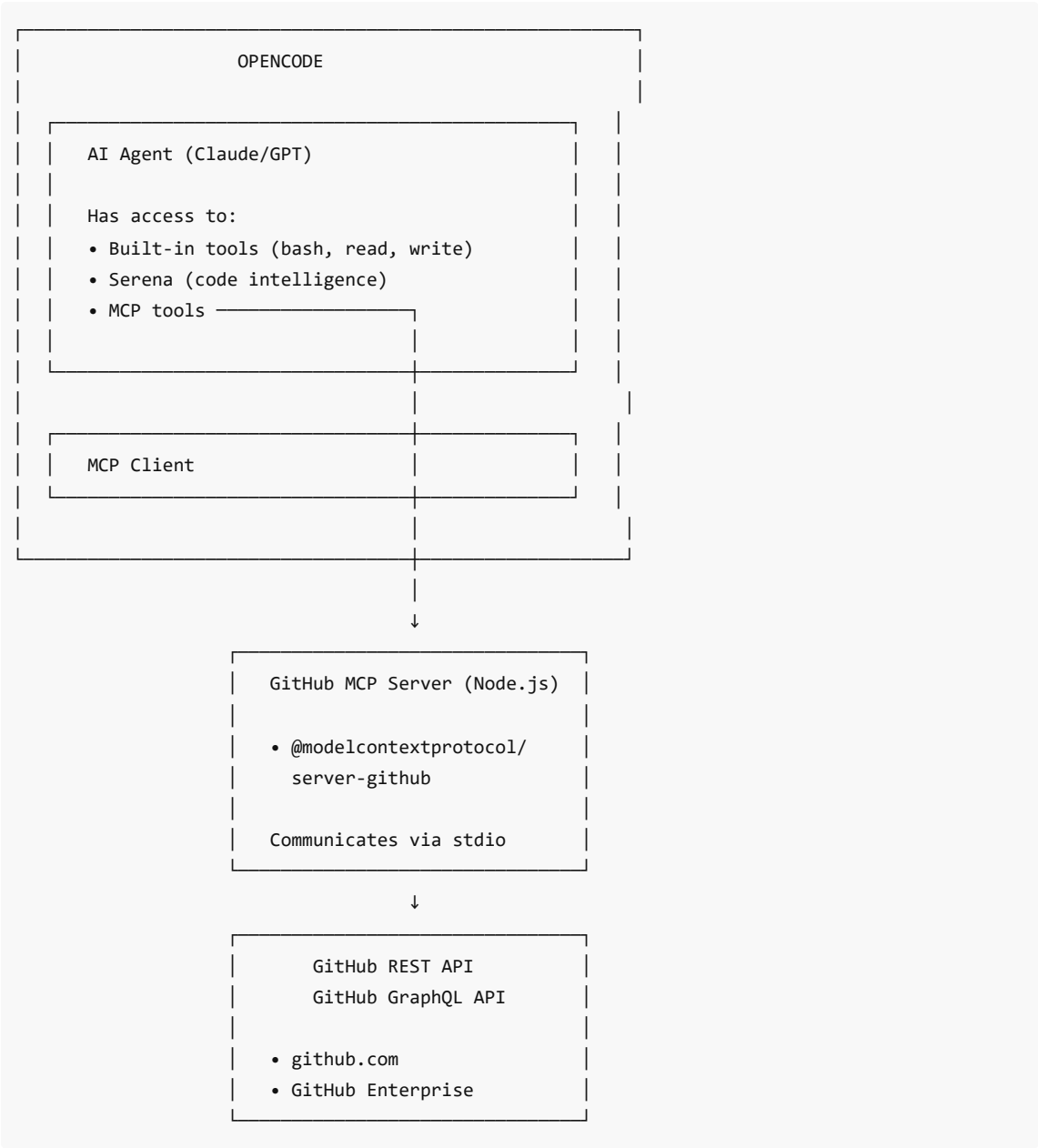
WITH GitHub MCP:

You: "Create a PR for this feature"

OpenCode: *Creates branch, pushes code, opens PR with description*

You: "Done! PR #123 is ready for review"

Architecture



What You'll Get

Available Capabilities

Once configured, you can use natural language to:

Issues & Projects

-  Create issues with templates
-  Search issues by label, assignee, state
-  Update issue status, labels, assignees
-  Add comments to issues

- ☒ Close/reopen issues
- ☒ Link issues to pull requests

Pull Requests

- ☒ Create PRs with auto-generated descriptions
- ☒ Review code and add comments
- ☒ Approve or request changes
- ☒ Merge PRs (merge, squash, rebase)
- ☒ Update PR descriptions and titles
- ☒ Check CI/CD status
- ☒ Resolve merge conflicts

Branches & Commits

- ☒ Create and delete branches
- ☒ List commits with filters
- ☒ View commit diffs
- ☒ Cherry-pick commits
- ☒ Compare branches

Code Search

- ☒ Search code across all repos
- ☒ Find functions, classes, patterns
- ☒ Search by language, path, repo
- ☒ Advanced search operators

Repositories

- ☒ Create new repositories
- ☒ Fork repositories
- ☒ List user/org repositories
- ☒ Update repository settings
- ☒ Manage topics and descriptions

Releases & Tags

- ☒ Create releases
- ☒ Upload release assets
- ☒ List releases and tags
- ☒ Update release notes

Collaboration

- ☒ Search users
- ☒ List collaborators
- ☒ Manage team access
- ☒ Request reviewers

Prerequisites

Required Software

1. **Node.js** (v18 or higher)

```
# Check your version
node --version # Should be v18.0.0 or higher
```

2. **npm** (comes with Node.js)

```
npm --version # Should be 9.0.0 or higher
```

3. **OpenCode** (v1.3.0 or higher)

```
opencode --version
```

Required Accounts

- **GitHub Account:** Personal or organization account
- **GitHub Personal Access Token:** With appropriate permissions

System Requirements

OS	Status	Notes
Windows	✔ Supported	Works on Windows 10/11
macOS	✔ Supported	Works on Intel and Apple Silicon
Linux	✔ Supported	All major distributions
WSL2	✔ Supported	Recommended for Windows developers

Installation Methods

Choose one of these methods based on your preference:

Method 1: NPM Global Install (Recommended)

Best for: Most users, easiest to maintain

```
# Install globally
npm install -g @modelcontextprotocol/server-github

# Verify installation
which mcp-server-github # macOS/Linux
where mcp-server-github # Windows
```

Pros:

- ✔ Simple one-line install
- ✔ Automatic PATH setup
- ✔ Easy to update
- ✔ Works from any directory

Cons:

- ❌ Requires admin/sudo on some systems
 - ❌ Global node_modules can get cluttered
-

Method 2: NPX (No Installation)

Best for: Testing, trying before committing, CI/CD

```
# No installation needed - npx runs on demand
# Configuration uses npx directly
```

Pros:

- ✅ No installation step
- ✅ Always uses latest version
- ✅ Clean, minimal footprint

Cons:

- ❌ Slower startup (downloads on first use)
 - ❌ Requires internet connection
 - ❌ Less control over version
-

Method 3: Local Project Install

Best for: Project-specific configuration, version pinning

```
# Navigate to your project
cd /path/to/your/project

# Install locally
npm install @modelcontextprotocol/server-github

# Creates node_modules/@modelcontextprotocol/server-github
```

Pros:

- ✅ Version pinned in package.json
- ✅ Team shares same version
- ✅ No global pollution

Cons:

- ❌ Must install per project
 - ❌ Larger project size
 - ❌ Path management in config
-

Method 4: From Source (Advanced)

Best for: Development, customization, contributing

```
# Clone the repository
git clone https://github.com/modelcontextprotocol/servers.git
```

```
cd servers/src/github

# Install dependencies
npm install

# Build
npm run build

# The built server is at: dist/index.js
```

Pros:

-  Full control
-  Can modify source
-  Latest unreleased features

Cons:

-  More complex setup
-  Manual updates
-  Build step required

Recommended Choice

For most users:

```
# Just run this:
npm install -g @modelcontextprotocol/server-github
```

This guide will use **Method 1 (Global Install)** for all examples.

Authentication Setup

The GitHub MCP server requires a Personal Access Token (PAT) for authentication.

Step 1: Create a GitHub Personal Access Token

Option A: Fine-Grained Personal Access Token (Recommended)

More secure, repository-specific permissions.

1. **Go to GitHub Settings:**

- Visit: <https://github.com/settings/tokens>
- Or: GitHub profile → Settings → Developer settings → Personal access tokens → Fine-grained tokens

2. **Click "Generate new token"**

3. **Configure the token:**

- **Token name:** OpenCode GitHub MCP
- **Expiration:** 90 days (or custom)
- **Description:** For OpenCode AI assistant GitHub integration

- **Repository access:** Choose one:
 - **All repositories:** If you want access to everything
 - **Only select repositories:** For specific repos only

4. Set Repository Permissions:

Required (minimum):

```
Contents: Read and Write
Pull requests: Read and Write
Issues: Read and Write
Metadata: Read-only (automatic)
```

Recommended (full functionality):

```
Actions: Read and Write
Administration: Read and Write
Checks: Read and Write
Commit statuses: Read and Write
Contents: Read and Write
Deployments: Read and Write
Discussions: Read and Write
Environments: Read and Write
Issues: Read and Write
Merge queues: Read and Write
Pull requests: Read and Write
Webhooks: Read and Write
Workflows: Read and Write
```

5. Set Account Permissions (optional):

```
Starring: Read and Write
Watching: Read and Write
```

6. Generate token and **COPY IT IMMEDIATELY** (you won't see it again!)

Option B: Classic Personal Access Token

Simpler but broader permissions.

1. Go to GitHub Settings:

- Visit: <https://github.com/settings/tokens>
- Or: GitHub profile → Settings → Developer settings → Personal access tokens → Tokens (classic)

2. Click "Generate new token (classic)"

3. Configure the token:

- **Note:** OpenCode GitHub MCP
- **Expiration:** 90 days (or custom)

4. Select Scopes:

Required (minimum):

- ✓ repo (Full control of private repositories)
- ✓ read:org (Read org and team membership)

Recommended (full functionality):

- ✓ repo (Full control of private repositories)
- ✓ workflow (Update GitHub Action workflows)
- ✓ write:packages (Upload packages to GitHub Package Registry)
- ✓ read:org (Read org and team membership)
- ✓ read:user (Read user profile data)
- ✓ user:email (Access user email addresses)

5. **Generate token** and **COPY IT** immediately!

Step 2: Store the Token Securely

Option A: Environment Variable (Recommended)

Windows (PowerShell):

```
# Set for current session
$env:GITHUB_PERSONAL_ACCESS_TOKEN = "ghp_your_token_here"

# Set permanently (User level)
[System.Environment]::SetEnvironmentVariable(
    'GITHUB_PERSONAL_ACCESS_TOKEN',
    'ghp_your_token_here',
    'User'
)

# Verify
echo $env:GITHUB_PERSONAL_ACCESS_TOKEN
```

Windows (CMD):

```
# Set for current session
set GITHub_PERSONAL_ACCESS_TOKEN=ghp_your_token_here

# Set permanently
setx GITHub_PERSONAL_ACCESS_TOKEN "ghp_your_token_here"

# Verify (restart CMD first)
echo %GITHub_PERSONAL_ACCESS_TOKEN%
```

macOS/Linux (bash/zsh):

```
# Add to ~/.bashrc or ~/.zshrc
echo 'export GITHUB_PERSONAL_ACCESS_TOKEN="ghp_your_token_here"' >> ~/.bashrc

# Or for zsh
```



```
echo 'export GITHUB_PERSONAL_ACCESS_TOKEN="ghp_your_token_here"' >> ~/.zshrc

# Reload shell
source ~/.bashrc # or source ~/.zshrc

# Verify
echo $GITHUB_PERSONAL_ACCESS_TOKEN
```

macOS (using keychain):

```
# Store in keychain
security add-generic-password \
  -a "$USER" \
  -s "GITHUB_PERSONAL_ACCESS_TOKEN" \
  -w "ghp_your_token_here"

# Retrieve from keychain
security find-generic-password \
  -a "$USER" \
  -s "GITHUB_PERSONAL_ACCESS_TOKEN" \
  -w
```

Option B: .env File (Project-specific)

```
# Create .env file in your project root
echo "GITHUB_PERSONAL_ACCESS_TOKEN=ghp_your_token_here" > .env

# Add to .gitignore (IMPORTANT!)
echo ".env" >> .gitignore
```

⚠️ **CRITICAL:** Never commit tokens to git!

Option C: Secure Credential Manager

Windows Credential Manager:

```
# Store credential
cmdkey /generic:"github-mcp-token" /user:"github" /pass:"ghp_your_token_here"

# Retrieve (for verification)
cmdkey /list | Select-String "github-mcp-token"
```

macOS Keychain (already covered above)

Linux Secret Service:

```
# Using secret-tool (GNOME Keyring)
secret-tool store --label='GitHub MCP Token' \
  service github-mcp \
  username github
```

```
# Retrieve
secret-tool lookup service github-mcp username github
```

Step 3: Verify Token Works

```
# Test the token
curl -H "Authorization: Bearer $GITHUB_PERSONAL_ACCESS_TOKEN" \
  https://api.github.com/user

# Expected: Your GitHub user info in JSON
```

If you see your username and profile data, the token works!

OpenCode Configuration

Now configure OpenCode to use the GitHub MCP server.

Configuration File Location

Choose one based on your needs:

Scope	Location	Use Case
Global	~/.config/opencode/opencode.json	All projects, all repos
Project	./opencode.json	Current project only
Custom	--config /path/to/config.json	Specific config file

Recommendation: Start with **global** configuration.

Method 1: Global Install Configuration

~/.config/opencode/opencode.json:

```
{
  "$schema": "https://opencode.ai/config.json",

  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      },
      "enabled": true
    }
  }
}
```

Explanation:

- `"type": "stdio"` - GitHub MCP uses standard input/output
 - `"command": "mcp-server-github"` - Assumes global install (in PATH)
 - `"{env:GITHUB_PERSONAL_ACCESS_TOKEN}"` - Reads from environment variable
 - `"enabled": true` - Activates the server
-

Method 2: NPX Configuration (No Install)

```
{
  "$schema": "https://opencode.ai/config.json",

  "mcp": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-github"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      },
      "enabled": true
    }
  }
}
```

Note: The `-y` flag auto-accepts the npx prompt.

Method 3: Local Install Configuration

```
{
  "$schema": "https://opencode.ai/config.json",

  "mcp": {
    "github": {
      "type": "stdio",
      "command": "node",
      "args": [
        "./node_modules/@modelcontextprotocol/server-github/dist/index.js"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      },
      "enabled": true
    }
  }
}
```

Method 4: From Source Configuration

```
{
  "$schema": "https://opencode.ai/config.json",

  "mcp": {
    "github": {
      "type": "stdio",
      "command": "node",
      "args": [
        "/path/to/servers/src/github/dist/index.js"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      },
      "enabled": true
    }
  }
}
```

Windows Path Considerations

If you have spaces in your paths:

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "C:\\Program Files\\nodejs\\node.exe",
      "args": [
        "C:\\Users\\YourUser\\AppData\\npm\\node_modules\\@modelcontextprotocol\\server-github\\dist\\index.js"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      },
      "enabled": true
    }
  }
}
```

Note: Use double backslashes `\\` in JSON.

Complete Configuration Example

Here's a full configuration with GitHub MCP and other settings:

~/config/opencode/opencode.json:

```

{
  "$schema": "https://opencode.ai/config.json",

  "// =====": "",
  "// GENERAL SETTINGS": "",
  "// =====": "",

  "model": "anthropic/claude-sonnet-4-5",
  "lsp": true,
  "autoupdate": true,
  "share": "manual",
  "snapshot": true,

  "// =====": "",
  "// MCP SERVERS": "",
  "// =====": "",

  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      },
      "enabled": true
    }
  },

  "// =====": "",
  "// FILE WATCHING": "",
  "// =====": "",

  "watcher": {
    "ignore": [
      "node_modules/**",
      "dist/**",
      "build/**",
      ".git/**",
      "*.log"
    ]
  }
}

```

Verification & Testing

Step 1: Validate Configuration

```

# Check if OpenCode can load the config
opencode --validate-config

```

```
# Expected: No errors
```

Step 2: List MCP Servers

```
# List all configured MCP servers
opencode mcp list

# Expected output:
# Available MCP servers:
#   • github (enabled)
```

Step 3: Test Connection

Start OpenCode and run these tests:

```
# Start OpenCode
opencode
```

Test 1: List tools

```
Show me available GitHub tools
```

Expected: You should see a list of GitHub-related tools like:

- github_create_issue
- github_create_pull_request
- github_search_code
- github_list_repositories
- etc.

Test 2: Get current user

```
Who am I on GitHub?
```

Expected: Your GitHub username and profile info

Test 3: List repositories

```
List my GitHub repositories
```

Expected: Your repositories listed

Test 4: Search code

```
Search for "console.log" in my repositories
```

Expected: Search results with file locations

Troubleshooting Connection Issues

If tests fail, check these:

Check 1: Token is Set

```
# Windows (PowerShell)
echo $env:GITHUB_PERSONAL_ACCESS_TOKEN

# Windows (CMD)
echo %GITHUB_PERSONAL_ACCESS_TOKEN%

# macOS/Linux
echo $GITHUB_PERSONAL_ACCESS_TOKEN
```

Should show: `ghp_xxxxxxxxxxxxx...`

Check 2: Server is Installed

```
# Check if command exists
which mcp-server-github # macOS/Linux
where mcp-server-github # Windows

# Or try running directly
mcp-server-github --version
```

Check 3: OpenCode Can Start Server

```
# Check OpenCode logs
tail -f ~/.local/share/opencode/logs/mcp.log # macOS/Linux
type %LOCALAPPDATA%\opencode\logs\mcp.log    # Windows
```

Look for errors related to GitHub MCP server startup.

Check 4: Token Permissions

```
# Test API access manually
curl -H "Authorization: Bearer $GITHUB_PERSONAL_ACCESS_TOKEN" \
  https://api.github.com/repos/OWNER/REPO/issues

# Replace OWNER/REPO with a real repository you have access to
```

If this fails, your token doesn't have the right permissions.

Available Tools

Once configured, you have access to these GitHub operations:

Issue Management

Tool	Description	Example Usage
github_create_issue	Create new issue	"Create an issue for the login bug"
github_list_issues	List issues	"Show open issues in my-repo"
github_get_issue	Get issue details	"Show me issue #42"
github_update_issue	Update issue	"Close issue #42 and add 'fixed' label"
github_add_issue_comment	Comment on issue	"Add comment to issue #42"
github_search_issues	Search issues	"Find issues labeled 'bug' in my repos"

Pull Request Management

Tool	Description	Example Usage
github_create_pull_request	Create PR	"Create PR for feature-branch"
github_list_pull_requests	List PRs	"Show open PRs in my-repo"
github_get_pull_request	Get PR details	"Show me PR #15"
github_update_pull_request	Update PR	"Update PR #15 title"
github_merge_pull_request	Merge PR	"Merge PR #15 with squash"
github_add_pull_request_comment	Comment on PR	"Add comment to PR #15"
github_create_pull_request_review	Review PR	"Approve PR #15"
github_search_pull_requests	Search PRs	"Find PRs by author:username"

Repository Management

Tool	Description	Example Usage
github_create_repository	Create repo	"Create repo called 'my-new-project'"
github_get_repository	Get repo info	"Show me repo details for my-repo"
github_list_repositories	List repos	"List my repositories"
github_search_repositories	Search repos	"Search for React projects"
github_fork_repository	Fork repo	"Fork owner/repo to my account"

Branch Management

Tool	Description	Example Usage
github_create_branch	Create branch	"Create branch feature/new-login"
github_list_branches	List branches	"Show branches in my-repo"

github_get_branch	Get branch info	"Show branch main"
-------------------	-----------------	--------------------

Commit & File Operations

Tool	Description	Example Usage
github_list_commits	List commits	"Show recent commits"
github_get_commit	Get commit details	"Show commit abc123"
github_get_file_contents	Read file	"Show README.md from my-repo"
github_create_or_update_file	Write file	"Update package.json in my-repo"
github_push_files	Push multiple files	"Push these 5 files to my-repo"

Search Operations

Tool	Description	Example Usage
github_search_code	Search code	"Find 'async function' in my repos"
github_search_users	Search users	"Find users with name John"

Release & Tag Management

Tool	Description	Example Usage
github_create_release	Create release	"Create v1.0.0 release"
github_list_releases	List releases	"Show releases for my-repo"
github_get_latest_release	Get latest release	"What's the latest release?"
github_list_tags	List tags	"Show tags in my-repo"

User & Team Operations

Tool	Description	Example Usage
github_get_me	Get current user	"Who am I on GitHub?"

Real-World Usage Examples

Example 1: Create Issue from Bug Report

You:

```
I found a bug: the login button doesn't work on mobile Safari.  
Create an issue in my-app repository with:  
- Title: "Login button broken on mobile Safari"  
- Labels: bug, high-priority  
- Assign to me
```

OpenCode:

I'll create that issue for you.

Creates issue using github_create_issue

✅ Created issue #127 in owner/my-app:
Title: "Login button broken on mobile Safari"
Labels: bug, high-priority
Assignee: @you
URL: <https://github.com/owner/my-app/issues/127>

Example 2: Create PR with AI-Generated Description

You:

I've finished the feature branch. Create a PR to main with a good description.

OpenCode:

Let me review your changes first...

Uses git diff to see changes

Analyzes code modifications

I see you've:

- Added user authentication
- Created login/logout endpoints
- Implemented JWT tokens
- Added tests

Creating PR...

Uses github_create_pull_request

✅ Created PR #45: "Add user authentication system"

Description:

Summary

Implements JWT-based authentication with login/logout endpoints.

Changes

- ✅ POST /api/login endpoint
- ✅ POST /api/logout endpoint
- ✅ JWT token generation and validation
- ✅ Protected route middleware
- ✅ Unit tests (95% coverage)

Testing

Run `npm test` - all tests passing

Ready for review!
URL: <https://github.com/owner/repo/pull/45>

Example 3: Search Codebase Across Repos

You:

Where do I use the old "fetchUser" function across all my repositories?

OpenCode:

Searching for "fetchUser" in your repositories...

Uses github_search_code

Found in 3 repositories:

- 📁 user-service (5 occurrences)
 - src/api/users.js:23
 - src/services/auth.js:45
 - tests/users.test.js:12
 - src/legacy/old-api.js:67
 - src/controllers/profile.js:34

- 📁 admin-panel (2 occurrences)
 - components/UserList.tsx:18
 - hooks/useUser.ts:7

- 📁 mobile-app (1 occurrence)
 - src/api/client.js:89

Would you like me to help refactor these to use the new "getUser" function?

Example 4: Review and Merge PR

You:

Review PR #42 in my-app and merge it if it looks good

OpenCode:

Let me review PR #42...

Uses github_get_pull_request to fetch PR details
Uses github_get_pull_request_files to see changed files
Uses github_get_pull_request_reviews to check existing reviews

PR #42: "Fix memory leak in WebSocket connection"

Changes:

- ✅ Properly closes WebSocket on component unmount

- ✓ Adds cleanup in useEffect
- ✓ Tests added and passing
- ✓ No merge conflicts
- ✓ 2 approvals from reviewers

Code looks good! All checks passed.

Uses github_merge_pull_request

- ✓ Merged PR #42 using squash merge
Commit: abc123 "Fix memory leak in WebSocket connection (#42)"

Example 5: Create Release

You:

Create a new release v2.1.0 with the recent changes

OpenCode:

Let me gather the changes since the last release...

Uses github_get_latest_release to find v2.0.0
Uses github_list_commits to get commits since v2.0.0
Generates changelog from commits

Creating release v2.1.0...

Uses github_create_release

- ✓ Created release v2.1.0

Release Notes

Features

- Added dark mode support (#123)
- Implemented CSV export (#125)
- Added keyboard shortcuts (#127)

Bug Fixes

- Fixed memory leak in WebSocket (#42)
- Resolved login redirect issue (#118)

Performance

- Optimized bundle size (-15%)
- Improved initial load time

URL: <https://github.com/owner/repo/releases/tag/v2.1.0>

Example 6: Find and Close Stale Issues

You:

```
Find issues in my-repo that haven't been updated in 6 months and close them as "stale"
```

OpenCode:

```
Searching for stale issues...

*Uses github_search_issues with date filter*

Found 8 stale issues (>6 months old):

1. #23 "Add export feature" (opened 8 months ago, no activity)
2. #45 "Improve performance" (opened 7 months ago, no activity)
3. #67 "Documentation updates" (opened 10 months ago, no activity)
...

Would you like me to:
1. Close all as "stale" with a comment?
2. Review each one first?
3. Cancel?
```

You: 1

OpenCode:

```
*For each issue, uses github_add_issue_comment then github_update_issue*

✅ Closed 8 stale issues with comment:
  "This issue has been automatically closed due to inactivity.
  Please reopen if this is still relevant."

Summary:
- Closed: #23, #45, #67, #78, #89, #92, #104, #115
- Added "stale" label to each
```

Example 7: Code Search and Refactor

You:

```
Find all uses of process.env.API_KEY in my repositories and create issues to move them to a
config file
```

OpenCode:

```
Searching for "process.env.API_KEY"...

*Uses github_search_code*

Found in 4 repositories:

📁 backend-api (3 occurrences)
```

📁 worker-service (2 occurrences)
📁 cron-jobs (1 occurrence)
📁 utils-library (1 occurrence)

Creating issues...

For each repo, uses github_create_issue

✅ Created 4 issues:

1. backend-api #89: "Move API_KEY to config file"
2. worker-service #23: "Move API_KEY to config file"
3. cron-jobs #45: "Move API_KEY to config file"
4. utils-library #12: "Move API_KEY to config file"

Each issue includes:

- File locations where API_KEY is used
- Suggested refactoring approach
- Security best practices link

Example 8: Multi-Repo Update

You:

Update the README.md in all my repos to include the new contributing guidelines

OpenCode:

Let me list your repositories first...

Uses github_list_repositories

Found 12 repositories. Checking which have README.md...

For each repo, uses github_get_file_contents

10 repositories have README.md. Would you like me to:

1. Update all 10 at once?
2. Show me what will change first?
3. Select specific repos?

You: 2

OpenCode:

Shows diffs for each README

I'll add a "Contributing" section to each README with:

- Link to CONTRIBUTING.md
- Code of conduct
- PR process
- Development setup

Ready to update all 10 repositories?

You: Yes

OpenCode:

For each repo, uses github_create_or_update_file

✅ Updated README.md in 10 repositories:

1. ✅ backend-api
2. ✅ frontend-app
3. ✅ mobile-app
- ...
10. ✅ docs-site

All changes committed to main branch.

Troubleshooting

Common Issues

Issue 1: "GitHub MCP server not found"

Symptoms:

Error: Cannot find module '@modelcontextprotocol/server-github'

Solution:

```
# Re-install globally
npm install -g @modelcontextprotocol/server-github

# Verify installation
which mcp-server-github # macOS/Linux
where mcp-server-github # Windows
```

Issue 2: "Authentication failed"

Symptoms:

Error: Bad credentials (401)

Solutions:

Check 1: Token is set

```
echo $GITHUB_PERSONAL_ACCESS_TOKEN # Should show ghp_xxx...
```

Check 2: Token is valid

```
curl -H "Authorization: Bearer $GITHUB_PERSONAL_ACCESS_TOKEN" \
https://api.github.com/user
```

Check 3: Token has correct permissions

- Go to <https://github.com/settings/tokens>
- Click on your token
- Verify it has `repo` , `read:org` , etc.

Check 4: Token not expired

- Check expiration date on GitHub

Fix: Create a new token with correct permissions.

Issue 3: "Rate limit exceeded"

Symptoms:

```
Error: API rate limit exceeded (403)
```

Solution:

```
# Check your rate limit status
curl -H "Authorization: Bearer $GITHUB_PERSONAL_ACCESS_TOKEN" \
https://api.github.com/rate_limit

# Shows:
# {
#   "rate": {
#     "limit": 5000,
#     "remaining": 0,
#     "reset": 1621234567 // Unix timestamp
#   }
# }
```

Fixes:

1. Wait until rate limit resets (check `reset` timestamp)
 2. Use authenticated token (5000/hour vs 60/hour)
 3. Reduce frequency of API calls
 4. Consider GitHub Enterprise for higher limits
-

Issue 4: "Server crashed or timeout"

Symptoms:

```
Error: MCP server github failed to start
Error: Timeout waiting for server response
```

Solutions:

Check 1: Server can run manually


```
# Try running server directly
GITHUB_PERSONAL_ACCESS_TOKEN=ghp_xxx mcp-server-github

# Should start without errors
```

Check 2: Node.js version

```
node --version # Should be v18+
```

Check 3: Check logs

```
# OpenCode MCP logs
cat ~/.local/share/opencode/logs/mcp.log # macOS/Linux
type %LOCALAPPDATA%\opencode\logs\mcp.log # Windows
```

Check 4: Increase timeout

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "timeout": 30000, // 30 seconds
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      }
    }
  }
}
```

Issue 5: "Permission denied to repository"

Symptoms:

```
Error: Not Found (404)
Error: Resource not accessible by integration (403)
```

Solutions:

1. Check repository access:

- Is the repository private?
- Does your token have access?
- Are you using fine-grained token with limited repo access?

2. Use correct repository reference:

- ✅ Correct: owner/repo
- ❌ Wrong: repo
- ❌ Wrong: https://github.com/owner/repo

3. Update token permissions:

- Go to token settings
- Add repository access
- Regenerate token if needed

Issue 6: "Cannot create PR: No commits between branches"

Symptoms:

Error: No commits between base and head branches

Solution:

```
# Check if your branch has commits
git log origin/main..HEAD

# If empty, you need to commit changes first
git add .
git commit -m "Your changes"
git push
```

Issue 7: "SSL/TLS certificate errors"

Symptoms:

Error: unable to verify the first certificate
Error: self signed certificate in certificate chain

Solutions:

For GitHub Enterprise with self-signed certs:

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}",
        "NODE_TLS_REJECT_UNAUTHORIZED": "0" // WARNING: Only for dev!
      }
    }
  }
}
```

⚠ **Security Warning:** Only use `NODE_TLS_REJECT_UNAUTHORIZED=0` in development with trusted networks.

Better solution: Install proper CA certificates.

Issue 8: "Works in terminal but not in OpenCode"

Symptoms:

- `curl` with token works
- Manual MCP server starts fine
- OpenCode can't connect

Solutions:

Check 1: Environment variables not loaded

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_hardcoded_token_here" // For testing only!
      }
    }
  }
}
```

If this works, the issue is environment variable loading.

Check 2: Restart OpenCode

```
# Fully quit and restart OpenCode
# Environment variables may need reload
```

Check 3: Use absolute paths

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "/usr/local/bin/mcp-server-github", // Absolute path
      // ...
    }
  }
}
```

Debug Mode

Enable debug logging:

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
```

```
    "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}",
    "DEBUG": "*" // Enable all debug logs
  },
  "enabled": true
}
},

"logLevel": "debug" // OpenCode debug logs
}
```

View logs:

```
# OpenCode logs
tail -f ~/.local/share/opencode/logs/*.log

# MCP server logs
tail -f ~/.local/share/opencode/logs/mcp-github.log
```

Best Practices

1. Token Management



DO:

- Use environment variables
- Use fine-grained tokens when possible
- Set reasonable expiration dates (90 days)
- Create separate tokens for different purposes
- Store in secure credential manager
- Rotate tokens regularly



DON'T:

- Hardcode tokens in config files
- Commit tokens to git
- Use tokens with excessive permissions
- Share tokens between services
- Use never-expiring tokens in production

2. Rate Limiting



DO:

- Be mindful of API rate limits (5000/hour authenticated)
- Cache results when appropriate
- Batch operations when possible
- Check rate limit status: `curl -H "Authorization: Bearer $TOKEN" https://api.github.com/rate_limit`



DON'T:

- Make unnecessary repeated calls

- Ignore rate limit errors
 - Create tight loops calling GitHub API
-

3. Error Handling

✓ DO:

- Check if operations succeed
- Handle 404 (not found) gracefully
- Handle 403 (permission denied)
- Retry on transient errors
- Provide clear error messages to user

✗ DON'T:

- Assume operations always succeed
 - Ignore error codes
 - Continue on critical failures
-

4. Security

✓ DO:

- Use HTTPS (default for github.com)
- Validate repository access before operations
- Use fine-grained tokens with minimal permissions
- Audit token usage periodically
- Revoke unused tokens

✗ DON'T:

- Disable SSL verification in production
 - Grant unnecessary permissions
 - Leave old tokens active
 - Use tokens in public logs
-

5. Performance

✓ DO:

- Use GraphQL for complex queries (when supported)
- Paginate large result sets
- Filter server-side when possible
- Use conditional requests (ETags)
- Cache frequently accessed data

✗ DON'T:

- Fetch all data when you need one item
 - Make serial calls that could be parallel
 - Ignore pagination
 - Request unnecessary fields
-

Advanced Configuration

GitHub Enterprise

For GitHub Enterprise Server:

```
{
  "mcp": {
    "github-enterprise": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GHE_TOKEN}",
        "GITHUB_API_URL": "https://github.enterprise.com/api/v3"
      },
      "enabled": true
    }
  }
}
```

Multiple GitHub Accounts

You can configure multiple GitHub MCP instances:

```
{
  "mcp": {
    "github-personal": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_TOKEN_PERSONAL}"
      },
      "enabled": true
    },
    "github-work": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_TOKEN_WORK}"
      },
      "enabled": true
    }
  }
}
```

Usage:

Using github-personal: List my personal repositories

Using github-work: List work repositories

Proxy Configuration

If behind a corporate proxy:

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}",
        "HTTP_PROXY": "http://proxy.company.com:8080",
        "HTTPS_PROXY": "http://proxy.company.com:8080",
        "NO_PROXY": "localhost,127.0.0.1"
      },
      "enabled": true
    }
  }
}
```

Custom Timeouts

For slow networks or large operations:

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "timeout": 60000, // 60 seconds (default: 30s)
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      },
      "enabled": true
    }
  }
}
```

Conditional Enabling

Enable only for specific projects:

Project-specific `opencode.json`:

```
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
```

```
    "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
  },
  "enabled": true // Only for this project
}
}
```

Global config: Set `"enabled": false` by default.

Security Considerations

1. Token Storage

Security Levels (Best to Worst):

1. **★★★ Secure Credential Manager**
 - macOS Keychain
 - Windows Credential Manager
 - Linux Secret Service
2. **★★ Environment Variables (User-level)**
 - Stored in shell RC files
 - Not visible to other users
 - Persists across sessions
3. **★ Environment Variables (Session-level)**
 - Set for current session only
 - Lost on session end
 - Good for temporary use
4. **✗ Project .env files**
 - Risk of accidental commit
 - Visible to anyone with file access
 - MUST be in .gitignore
5. **✗✗ Hardcoded in config**
 - NEVER do this
 - Will be committed to git
 - Visible in version history

2. Token Permissions

Principle of Least Privilege:

Only grant permissions you actually need:

Need to read issues?

- ✅ Grant: `issues:read`
- ✗ Don't grant: `repo` (full access)

Audit Checklist:

- ☐ Can read public repositories
 - ☐ Can read private repositories (if needed)
 - ☐ Can create issues (if needed)
 - ☐ Can create PRs (if needed)
 - ☐ Can merge PRs (if needed)
 - ☐ Can delete repos? (probably NO!)
 - ☐ Can manage org settings? (probably NO!)
-

3. Token Rotation

Recommended Schedule:

- Personal projects: Every 90 days
- Team projects: Every 60 days
- Production systems: Every 30 days

Rotation Process:

1. Create new token with same permissions
 2. Update environment variable
 3. Test new token works
 4. Revoke old token
 5. Update documentation
-

4. Monitoring

What to Monitor:

- API rate limit usage
- Failed authentication attempts
- Unusual access patterns
- Token usage from unexpected locations

How to Monitor:

```
# Check rate limit
curl -H "Authorization: Bearer $TOKEN" \
  https://api.github.com/rate_limit

# View your tokens
# https://github.com/settings/tokens

# Check for security alerts
# https://github.com/settings/security
```

5. Incident Response

If token is compromised:

1. **Immediate** (within 5 minutes):

- Revoke token at <https://github.com/settings/tokens>
- Remove from environment variables
- Remove from config files

2. **Short-term** (within 1 hour):

- Generate new token
- Update configurations
- Audit recent API calls
- Check for unauthorized changes

3. **Follow-up** (within 24 hours):

- Review security logs
- Update documentation
- Implement better storage
- Consider enabling 2FA

FAQ

Q: Can I use SSH keys instead of Personal Access Tokens?

A: No. The GitHub MCP server uses the GitHub REST/GraphQL APIs, which require HTTP authentication (Personal Access Tokens). SSH is only for git operations.

Q: Does this work with GitHub Enterprise?

A: Yes! Set the `GITHUB_API_URL` environment variable:

```
"env": {  
  "GITHUB_API_URL": "https://github.enterprise.com/api/v3",  
  "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:TOKEN}"  
}
```

Q: Can I use GitHub App authentication?

A: The MCP server currently supports Personal Access Tokens. GitHub App authentication is not yet supported.

Q: Will this push commits without asking me?

A: OpenCode will always explain what it's going to do before executing operations. You can review and approve/reject. Configure confirmation requirements in your OpenCode settings.

Q: Can I use this for organization repositories?

A: Yes, if your token has access to the organization. For fine-grained tokens, you'll need to grant organization access.

Q: Does this work offline?

A: No. The GitHub MCP server requires internet access to communicate with GitHub APIs.

Q: Can I limit which repositories OpenCode can access?

A: Yes! Use fine-grained Personal Access Tokens and select only specific repositories.

Q: How do I update the GitHub MCP server?

```
# For global install
npm update -g @modelcontextprotocol/server-github

# Check version
npm list -g @modelcontextprotocol/server-github
```

Q: Can I use this with GitLab or Bitbucket?

A: No, this is specific to GitHub. However, there are separate MCP servers for GitLab and other platforms. Check the MCP server registry.

Q: What's the difference between this and the `gh` CLI?

A:

- `gh` CLI: Direct command-line tool for GitHub
 - **GitHub MCP**: Integration layer for AI assistants like OpenCode
 - **You can use both** - they don't conflict
-

Q: Can I create custom GitHub MCP tools?

A: The GitHub MCP server provides a standard set of tools. For custom functionality, you'd need to create your own MCP server or use the GitHub APIs directly via OpenCode's bash tool.

Q: Is my code sent to GitHub through this?

A: Only when you explicitly perform operations (create PR, push files, etc.). OpenCode's local operations don't send code to GitHub.

Q: Can I use multiple tokens for different repos?

A: Configure multiple GitHub MCP instances with different tokens (see [Multiple GitHub Accounts](#) section).

Q: What happens if my rate limit is exceeded?

A: Operations will fail with a rate limit error. Wait for the rate limit to reset (check reset time) or upgrade to GitHub Pro/Enterprise for higher limits.

Summary Checklist

Installation 

- ☐ Node.js v18+ installed
- ☐ GitHub MCP server installed: `npm install -g @modelcontextprotocol/server-github`
- ☐ Installation verified: `mcp-server-github --version`

Authentication

- ☐ Personal Access Token created on GitHub
- ☐ Token has required permissions (repo, read:org minimum)
- ☐ Token stored securely in environment variable
- ☐ Token tested: `curl -H "Authorization: Bearer $TOKEN" https://api.github.com/user`

Configuration

- ☐ OpenCode config file created/updated
- ☐ MCP server configured with correct command
- ☐ Environment variable referenced correctly
- ☐ Server enabled: `"enabled": true`

Verification

- ☐ Config validated: `opencode --validate-config`
- ☐ Server listed: `opencode mcp list`
- ☐ Tools available in OpenCode session
- ☐ Test operation successful (e.g., list repos)

Security

- ☐ Token not hardcoded in config
- ☐ Token not committed to git
- ☐ Minimal permissions granted
- ☐ Token expiration set (90 days recommended)
- ☐ .gitignore includes sensitive files

Quick Start (TL;DR)

For the impatient, here's the fastest path:

```
# 1. Install
npm install -g @modelcontextprotocol/server-github

# 2. Create token at https://github.com/settings/tokens
#   Grant: repo, read:org

# 3. Set environment variable (pick your shell)
# PowerShell:
$env:GITHUB_PERSONAL_ACCESS_TOKEN = "ghp_your_token"
# Bash/Zsh:
export GITHUB_PERSONAL_ACCESS_TOKEN="ghp_your_token"

# 4. Configure OpenCode
```

```
echo '{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "mcp-server-github",
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "{env:GITHUB_PERSONAL_ACCESS_TOKEN}"
      },
      "enabled": true
    }
  }
}' > ~/.config/opencode/opencode.json

# 5. Test
opencode
# Then type: "List my GitHub repositories"
```

Done! 🎉

Additional Resources

Official Documentation

- **GitHub MCP Server:** <https://github.com/modelcontextprotocol/servers/tree/main/src/github>
- **MCP Specification:** <https://modelcontextprotocol.io/>
- **OpenCode Docs:** <https://opencode.ai/docs>
- **GitHub API:** <https://docs.github.com/en/rest>

Community

- **OpenCode Discord:** <https://discord.gg/opencode>
- **GitHub Discussions:** <https://github.com/modelcontextprotocol/servers/discussions>
- **Stack Overflow:** Tag `opencode` or `model-context-protocol`

Related Guides

- **OpenCode Installation Guide Enhanced** - How to install OpenCode
- **Serena Code Intelligence Guide** - Using OpenCode's built-in code tools
- **Serena NOT an MCP Guide** - Understanding built-in vs MCP tools

Document Information:

- **Version:** 1.0
- **Last Updated:** May 15, 2026
- **OpenCode Compatibility:** 1.3.0+
- **GitHub MCP Version:** Latest (check npm)
- **Author:** OpenCode Documentation Team

Happy coding with GitHub and OpenCode! 🚀

If you have questions or run into issues, refer to the [Troubleshooting](#) section or ask in the OpenCode community.